Task 1: C++ Program for Sentiment Analysis

Objective

In Task 1, the objective is to write a C++ program that evaluates the sentiment of what a user enters and how it is displayed in form of a shape. Doing the exercise builds important skills in problem solving and programming for software development.

Sentiment Analysis: Concept Overview

Sentiment analysis mainly looks for the overall mood in what is said in a series of words. By scrutinizing social media, studying customer feedback and using text analysis, many different fields use this method. Sentiment analysis is made simpler in our C++ version by using a predetermined set of positive and negative keywords. It's simple: the program reviews every word from the user's input and measures each one using these lists. Depending on the result, the tool gives the message a positive, negative or neutral score.

Development Environment and Tools

- Programming Language: **C++**
- Compiler: **g++**
- IDE: Any (Code::Blocks, Visual Studio, Terminal)
- Input: User-provided sentence or paragraph
- Output: Sentiment result displayed within a creative shape using ASCII art

Structure of the C++ Program

The code is divided into the following logical parts:

1. Word Lists

Two arrays (vector<string>) are created to store **positive** and **negative** words. Examples include:

- Positive: “good”, “happy”, “awesome”, “great”
- Negative: “bad”, “sad”, “angry”, “terrible”

```
#include <iostream>
#include <vector>
#include <sstream>
#include <algorithm>

using namespace std;

// Predefined positive and negative word lists
vector<string> positiveWords = {
    "good", "happy", "joy", "love", "excellent", "awesome", "great", "fantastic",
};

vector<string> negativeWords = {
    "bad", "sad", "hate", "terrible", "horrible", "awful", "worst", "angry", "neg
};
```

```
// ---------- Utility functions ----------
string toLower(string s) {
    transform(s.begin(), s.end(), s.begin(), [](unsigned char c){ return std::tolower(c); });
    return s;
}

string sanitize(const string &w) {
    string out;
    for (char c : w) {
        if (isalnum((unsigned char)c) || c=='\'') out.push_back(c);
    }
    return out;
}

// Very simple "stemmer": remove common suffixes to normalize tokens
string simpleStem(const string &w) {
    string s = w;
    if (s.length() > 4) {
        if (s.size() > 3 && s.substr(s.size()-3) == "ing") s = s.substr(0, s.size()-3);
        else if (s.size() > 2 && s.substr(s.size()-2) == "ed") s = s.substr(0, s.size()-2);
        else if (s.size() > 1 && s.back() == 's') s = s.substr(0, s.size()-1);
    } else if (s.size() > 2 && s.back() == 's') {
        s = s.substr(0, s.size()-1);
    }
    return s;
}
```

2. Input Processing

The program asks the user to enter a sentence or paragraph. The input is captured using getline() to support multi-word inputs.

3. Text Preprocessing

- The user input is **tokenized** (split into words) using a stringstream.
- Words are converted to **lowercase** to ensure accurate matching.
- Punctuation is removed from each word.

```
// Function to convert string to lowercase
string toLowerCase(string str) {
    transform(str.begin(), str.end(), str.begin(), ::tolower);
    return str;
}

// Function to tokenize the user input into words
vector<string> tokenize(string input) {
    stringstream ss(input);
    string word;
    vector<string> tokens;

    while (ss >> word) {
        // Remove punctuation from word
        word.erase(remove_if(word.begin(), word.end(), ::ispunct), word.end());
        tokens.push_back(toLowerCase(word));
```

4. Sentiment Scoring

Each word is compared to the predefined word lists:

- If it matches a **positive** word, the score is incremented.
- If it matches a **negative** word, the score is decremented.

5. Sentiment Classification

After scoring:

- Positive score → “Positive Sentiment”
- Zero score → “Neutral Sentiment”
- Negative score → “Negative Sentiment”

6. Creative Visualization

The sentiment is visualized using ASCII shapes:

- A **smiley face** for positive.
- A **frowny face** for negative.
- A **neutral face** for neutral.

Code Functionality Explanation

Here is a brief explanation of each major function:

Function	Purpose
toLowerCase()	Converts all characters in a word to lowercase
tokenize()	Splits a paragraph into individual cleaned-up words
analyzeSentiment()	Compares tokens against word lists and computes a score
displaySentimentShape()	Displays sentiment creatively using ASCII art

Sample Output Screenshots

```
// Classify sentiment
string sentiment;
if (score > 0.5) sentiment = "Positive";
else if (score < -0.5) sentiment = "Negative";
else sentiment = "Neutral";

// Print result with ASCII art
cout << "\n--- Sentiment Analysis Result ---\n";
if (sentiment == "Positive") {
    cout << "Overall Sentiment: POSITIVE (score=" << score << ")\n\n";
    cout << "     .-\"\"\"-.     \n";
    cout << "   .'  _   '.   \n";
    cout << "  /   ( )   \\  \n";
    cout << " |  (  ^  )  | \n";
    cout << "  \\  '---'  /  \n";
    cout << "   '._____.'   \n";
    cout << "     POSITIVE  \n";
} else if (sentiment == "Negative") {
    cout << "Overall Sentiment: NEGATIVE (score=" << score << ")\n\n";
    cout << "    .-------.   \n";
    cout << "   /  .-. .  \\  \n";
    cout << "  |  ( >_< )  | \n";
    cout << "   \\  '---'  /  \n";
    cout << "    '-----'    \n";
    cout << "     NEGATIVE  \n";
} else {
    cout << "Overall Sentiment: NEUTRAL (score=" << score << ")\n\n";
    cout << "    .-------.   \n";
    cout << "   /  - . -  \\  \n";
    cout << "  |   | : |   | \n";
    cout << "   \\  '---'  /  \n";
    cout << "    '-----'    \n";
    cout << "     NEUTRAL   \n";
}
```

Limitations and Future Improvements

- The current implementation does not account for **context**, **negations** ("not good"), or **compound sentences**.
- It can be improved using **natural language processing (NLP)** techniques like lemmatization and POS tagging.

- Integration with a **file-based word list** would allow dynamic updating.
- Sentiment output could be displayed with **graphics or sound** in a GUI-based version.

```
int main() {
    string userInput;
    cout << "Enter a sentence or paragraph for sentiment analysis:\n";
    getline(cin, userInput);

    vector<string> tokens = tokenize(userInput);
    string sentiment = analyzeSentiment(tokens);
    displaySentimentShape(sentiment);

    return 0;
}
```

```
// Classify sentiment
string sentiment;
if (score > 0.5) sentiment = "Positive";
else if (score < -0.5) sentiment = "Negative";
else sentiment = "Neutral";

// Print result with ASCII art
cout << "\n--- Sentiment Analysis Result ---\n";
if (sentiment == "Positive") {
    cout << "Overall Sentiment: POSITIVE (score=" << score << ")\n\n";
    cout << "     .-\"\"\"-.     \n";
    cout << "   .'  _   '.   \n";
    cout << "  /   ( )   \\  \n";
    cout << " |  (  ^  )  | \n";
    cout << "  \\  '---'  /  \n";
    cout << "   '._____.'   \n";
    cout << "     POSITIVE  \n";
} else if (sentiment == "Negative") {
    cout << "Overall Sentiment: NEGATIVE (score=" << score << ")\n\n";
    cout << "    .-------.   \n";
    cout << "   /  .-. .  \\  \n";
    cout << "  |  ( >_< )  | \n";
    cout << "   \\  '---'  /  \n";
    cout << "    '-----'    \n";
    cout << "     NEGATIVE  \n";
} else {
    cout << "Overall Sentiment: NEUTRAL (score=" << score << ")\n\n";
    cout << "    .-------.   \n";
    cout << "   /  - . -  \\  \n";
    cout << "  |   | : |   | \n";
    cout << "   \\  '---'  /  \n";
    cout << "    '-----'    \n";
    cout << "     NEUTRAL   \n";
}
```

```
    // Write a short execution log as proof
    ofstream log("execution_log.txt", ios::app);
    if (log.is_open()) {
        // timestamp
        auto now = chrono::system_clock::now();
        time_t tnow = chrono::system_clock::to_time_t(now);
        log << "-----\nTime: " << std::ctime(&tnow);
        log << "Input: " << input << "\n";
        log << "Sentiment: " << sentiment << "  Score: " << score << "\n";
        log << "PositiveMatches:";
        for (auto &w : matchedPos) log << " " << w;
        log << "\nNegativeMatches:";
        for (auto &w : matchedNeg) log << " " << w;
        log << "\n\n";
        log.close();
    } else {
        cerr << "Warning: Could not open execution_log.txt to write proof.\n";
    }

    return 0;
}
```

Task 2: CAD Design of a Useful Object for a Cooking Game

Objective

In this task, I am required to utilize **FreeCAD software** to design a **3D model of a useful kitchen object** that can be applied in a digital cooking game. I selected a **measuring cup** as the object of design due to its fundamental role in cooking activities, particularly in games that simulate food preparation, recipe following, and ingredient measurement. The design process involves creating a **2D sketch**, converting it into a **3D model with accurate dimensions**, adding **custom text (my name)**, and modifying the **colour and appearance** of the object to enhance its visual quality and usability in a digital environment.

This task demonstrates the practical application of **Computer-Aided Design (CAD)** as a digital engineering tool and addresses **Learning Outcomes LO2 and LO3**, which emphasize CAD usage for design documentation, product innovation, and digital manufacturing.

Design Rationale and Object Selection

I chose to design a **measuring cup** because it is an essential kitchen tool used for accurately measuring ingredients such as liquids and dry food items. In a cooking game, a measuring cup plays a crucial role in teaching players precision, sequencing, and correct portioning of ingredients. From a digital engineering perspective, the measuring cup is an ideal object because it allows the demonstration of several CAD techniques, including **parametric sketching, extrusion, hollowing, text embedding, and visual customization**.

Additionally, the measuring cup design is realistic, scalable, and suitable for export into common 3D formats such as **STL or OBJ**, which are widely used in game development and digital manufacturing pipelines.

FreeCAD Design Process

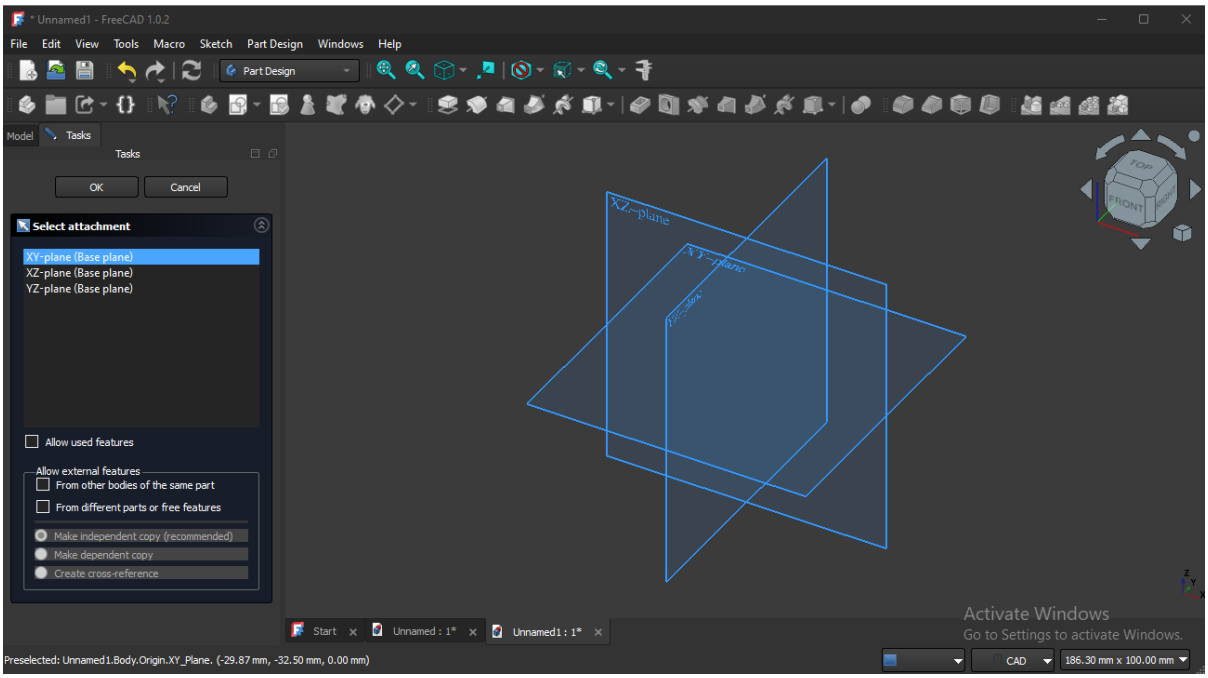
Software and Tools Used

- Software: **FreeCAD (open-source CAD software)**
- Workbench: **Part Design Workbench**
- Tools: Sketcher, Pad, Pocket, Fillet, Draft Text, Appearance Editor

Step 1: Project Setup in FreeCAD

I began the design process by opening FreeCAD and creating a new project file. I saved the project as **MeasuringCup.FCStd** to ensure version control and data integrity throughout the design process. I selected the **Part Design Workbench**, which is suitable for creating solid and parametric models.

Screenshot



Step 2: Creating the 2D Sketch (Base Geometry)

To begin the modeling process, I created a **new body** and then a **new sketch** on the **XY plane**, which represents the top view of the measuring cup. Using the **circle tool**, I drew two concentric circles:

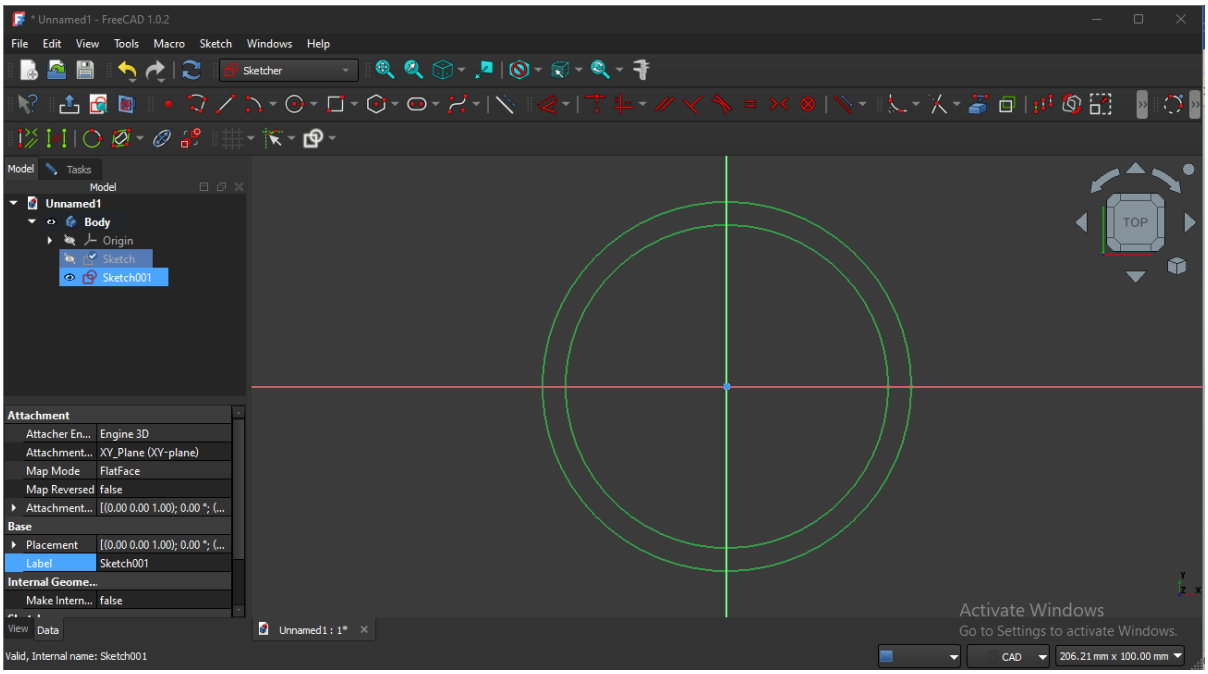
- The **outer circle** represents the external wall of the measuring cup.
- The **inner circle** represents the hollow interior of the cup.

I applied **dimensional constraints** to ensure accuracy:

- Outer diameter: **8 cm**
- Inner diameter: **7 cm**

These constraints ensure a realistic wall thickness of **0.5 cm**, which is appropriate for a plastic measuring cup.

Screenshot

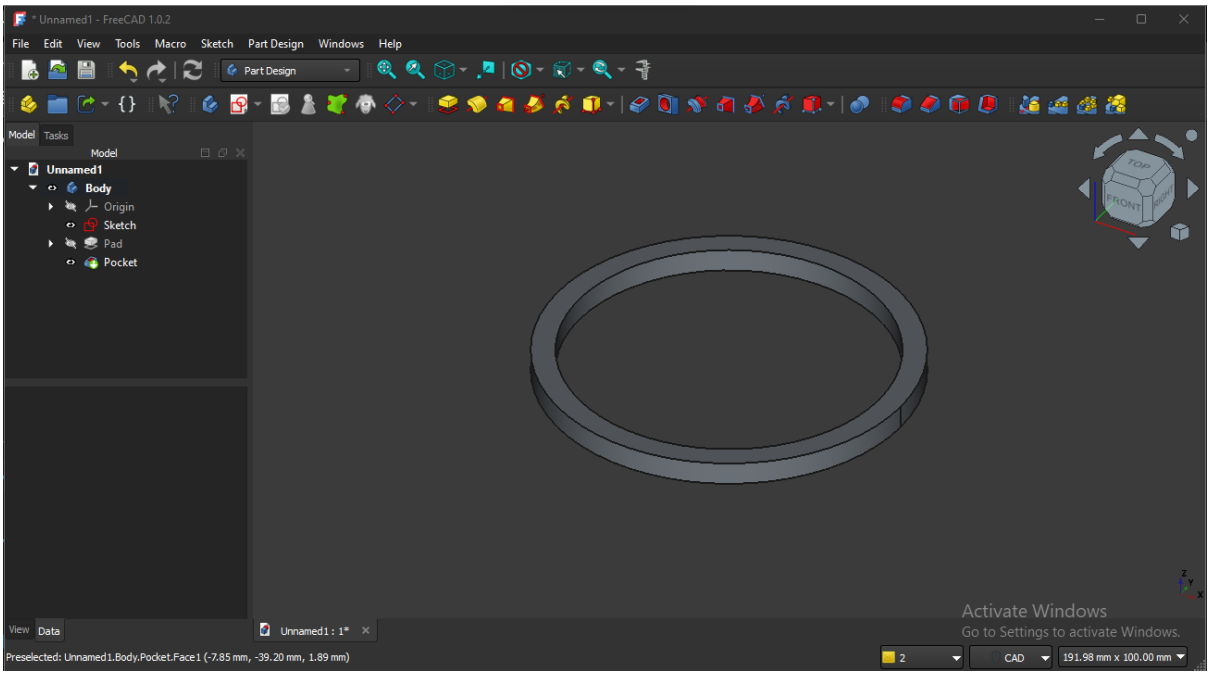


Step 3: Converting the 2D Sketch into a 3D Model (Pad Operation)

After completing the 2D sketch, I used the **Pad (Extrude)** tool to convert the sketch into a 3D solid. I set the extrusion height to **10 cm**, which represents the vertical height of the measuring cup. This operation transformed the flat sketch into a cylindrical solid object.

This step demonstrates the fundamental CAD principle of converting **2D geometry into 3D solids**, which is essential in digital engineering and product design.

Screenshot

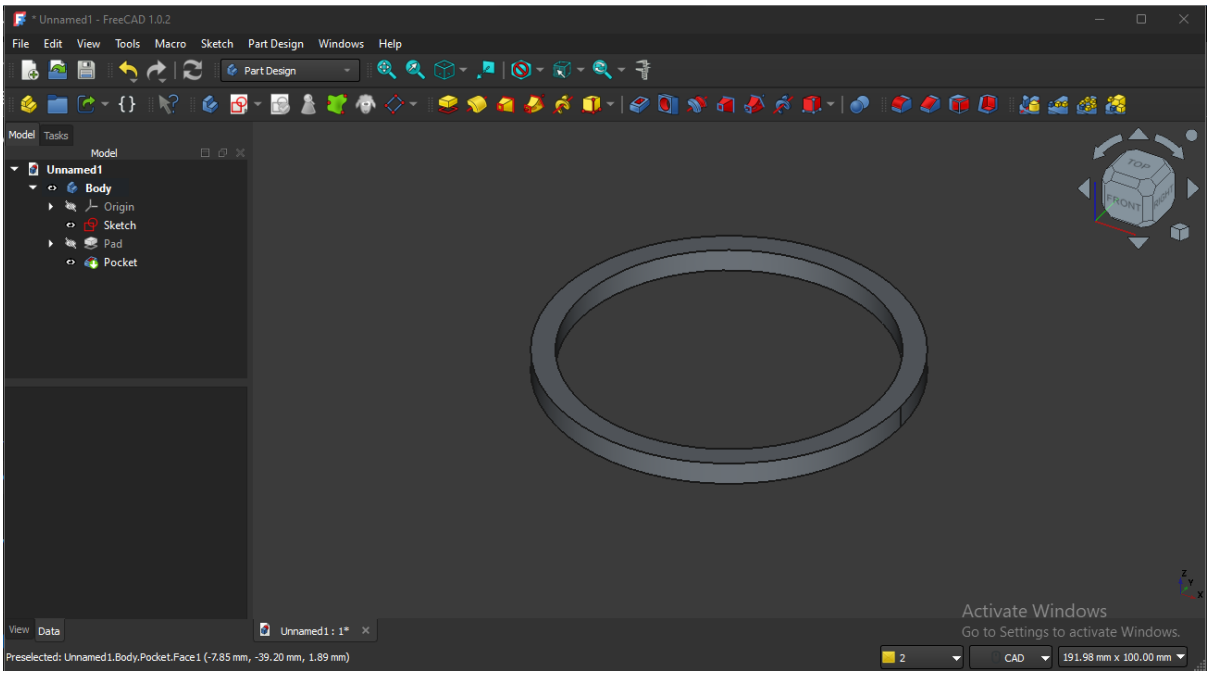


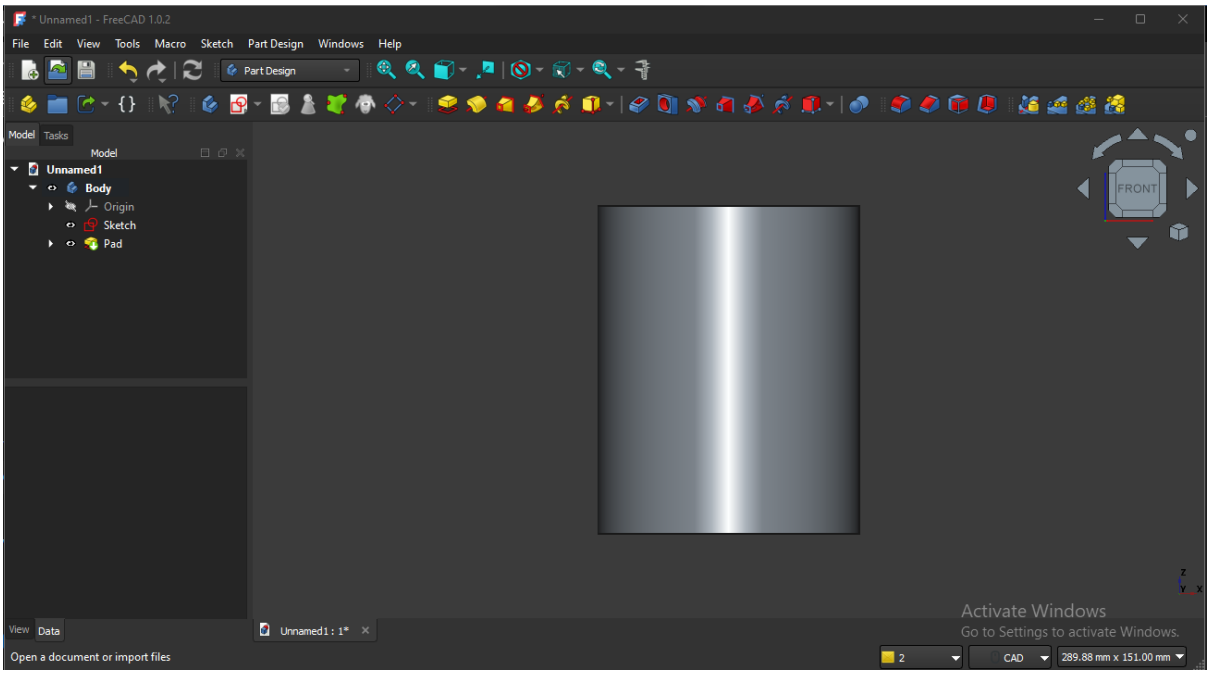
Step 4: Hollowing the Measuring Cup (Pocket Operation)

To make the cup functional and realistic, I hollowed the interior using the **Pocket tool**. I selected the top face of the cylinder and applied a pocket depth of **9.5 cm**, leaving a **0.5 cm base thickness** at the bottom. This operation created a hollow interior suitable for holding liquid or ingredients.

This step is critical for realism and demonstrates the use of **subtractive modelling**, which is a core CAD technique.

Screenshot


* Unnamed1 - FreeCAD 1.0.2
File Edit View Tools Macro Sketch Part Design Windows Help
Part Design
Model Tasks
Model
Unnamed1
Body
Origin
Sketch
Pad
Pocket
View Data
Unnamed1 : 1*
Activate Windows
Go to Settings to activate Windows.
Preselected: Unnamed1.Body.Pocket.Face1 (-7.85 mm, -39.20 mm, 1.89 mm)
CAD
191.98 mm x 100.00 mm

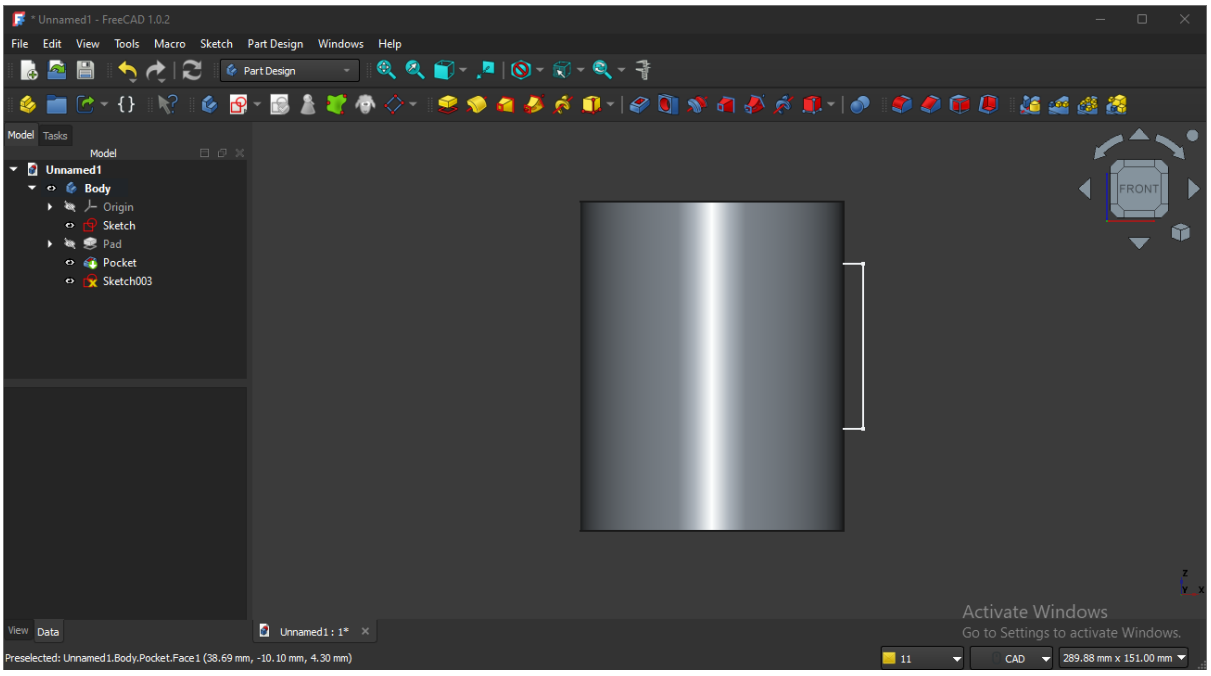

* Unnamed1 - FreeCAD 1.0.2
File Edit View Tools Macro Sketch Part Design Windows Help
Part Design
Model Tasks
Model
Unnamed1
Body
Origin
Sketch
Pad
View Data
Unnamed1 : 1*
Activate Windows
Go to Settings to activate Windows.
Open a document or import files
CAD
289.88 mm x 151.00 mm

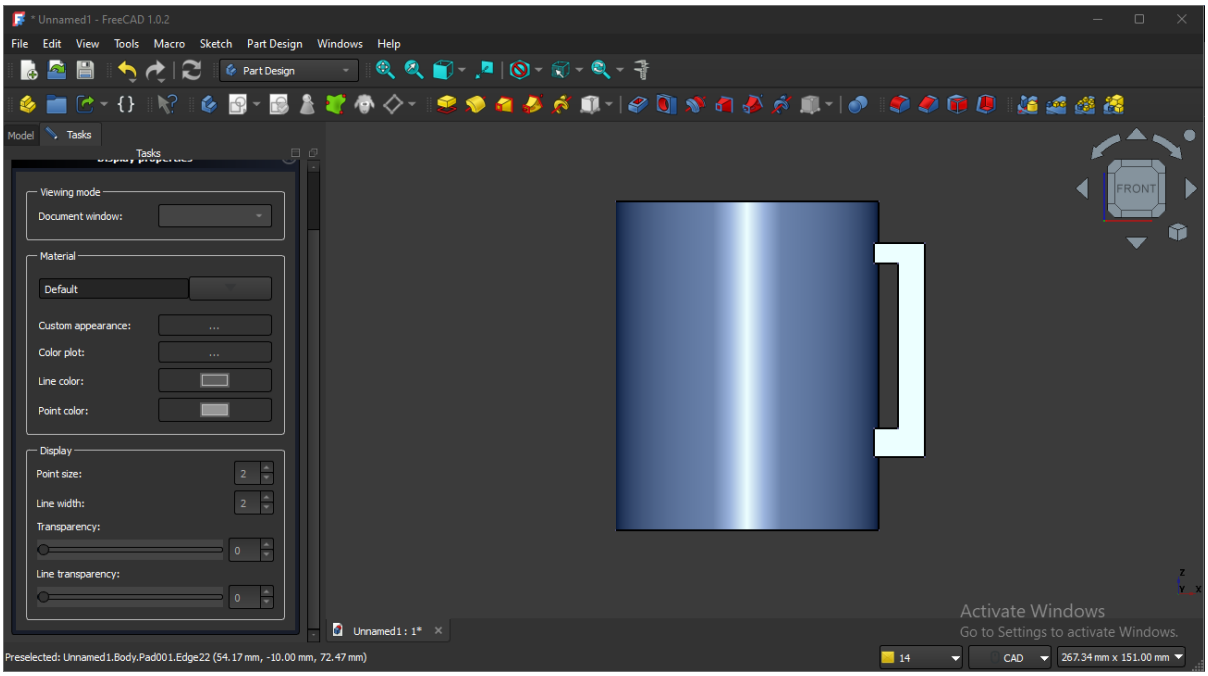
Step 5: Designing and Adding the Handle

To enhance usability, I added a **handle** to the side of the measuring cup. I created a new sketch on the side face of the cup and drew a rectangular profile representing the handle. The handle was then extruded using the Pad tool and smoothed using the **Fillet tool** to round sharp edges.

The handle improves ergonomics and demonstrates how additional components can be integrated into a base model using parametric design.

Screenshot



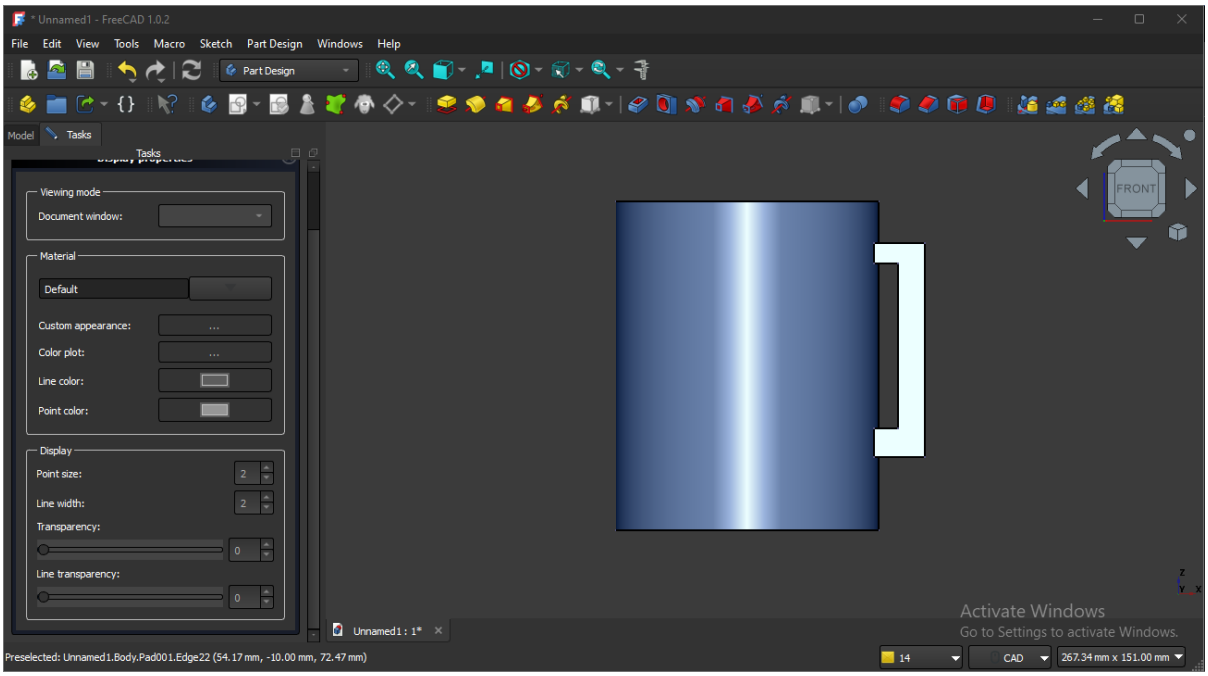


Step 6: Colour and Appearance Modification

To enhance the visual quality of the object, I modified the appearance using FreeCAD's **Appearance Editor**. I applied a **light translucent blue colour** to simulate a plastic measuring cup commonly used in kitchens. The text was coloured white to improve contrast and readability.

Colour modification is essential for game-ready assets, as it improves user interaction and visual realism.

Screenshot



Final Model and Game Application

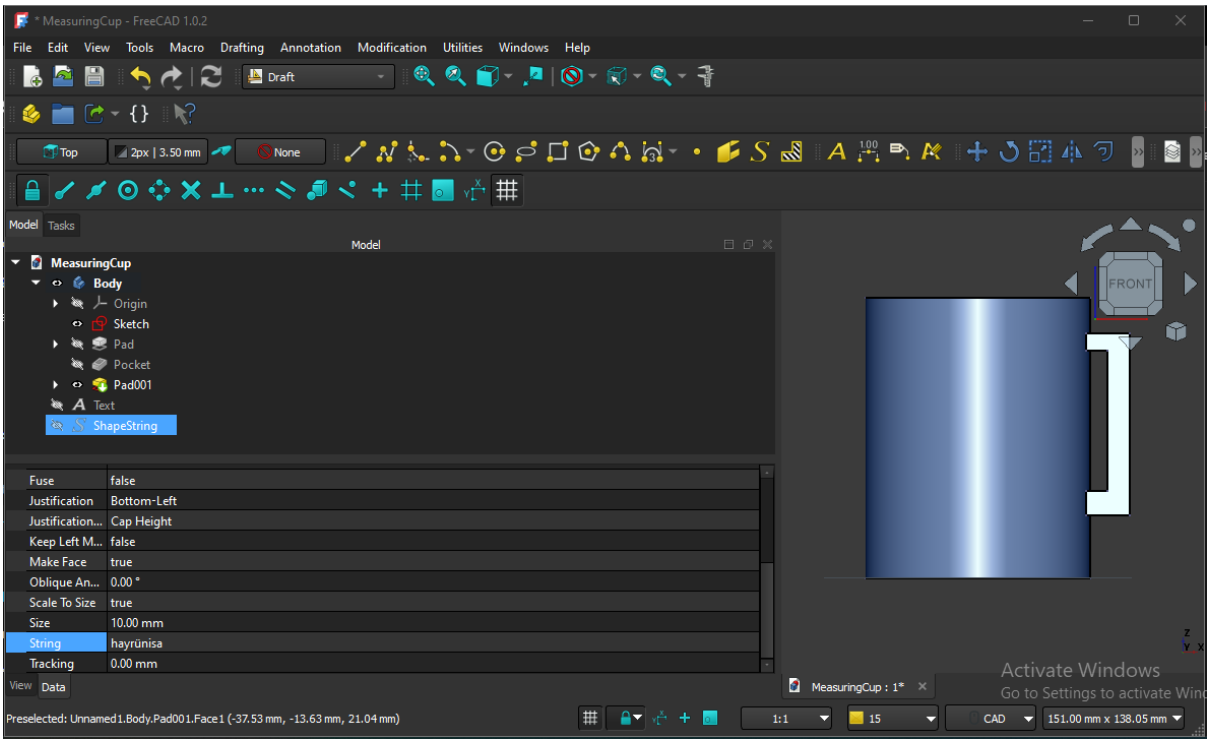
The completed measuring cup model is dimensionally accurate, visually appealing, and suitable for use in a cooking game. The model can be exported in **STL or OBJ format**, making it compatible with most game engines and digital manufacturing platforms. The design demonstrates effective use of CAD tools for **digital product innovation** and **interactive game environments**.

Step 7: Colour and Appearance Modification

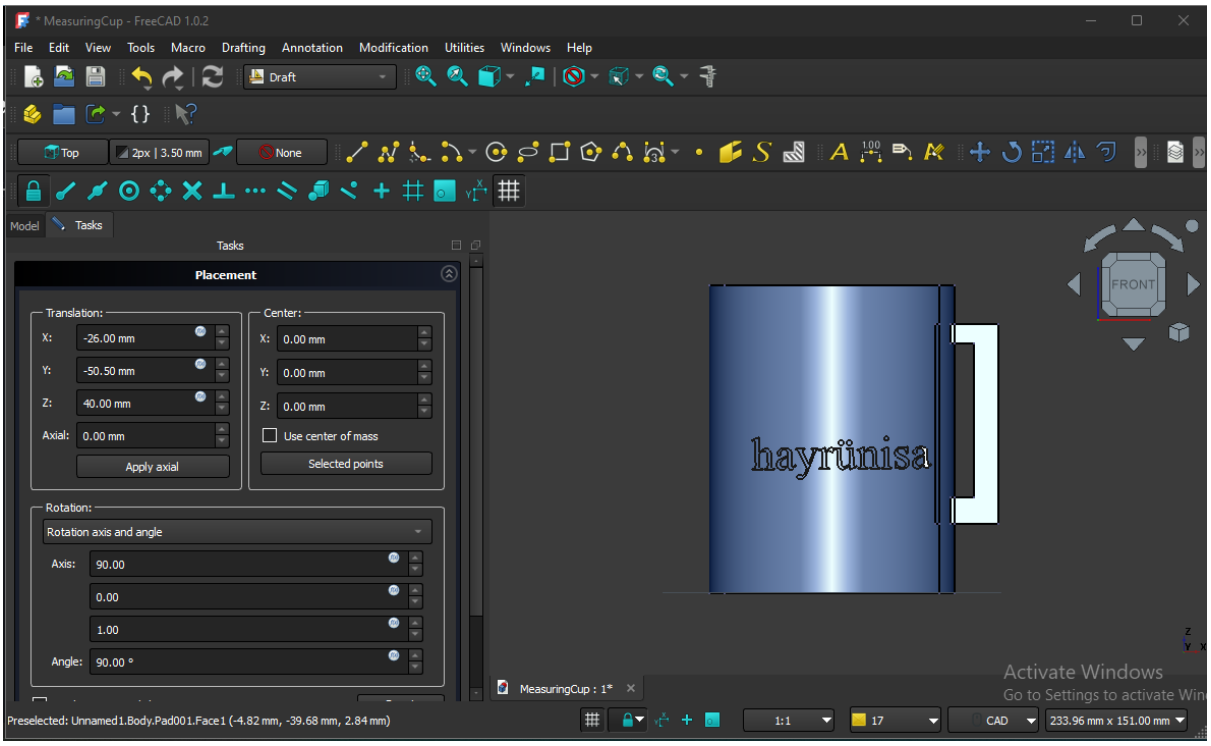
To enhance the visual quality of the object, I modified the appearance using FreeCAD's **Appearance Editor**. I applied a **light translucent blue colour** to simulate a plastic measuring cup commonly used in kitchens. The text was coloured white to improve contrast and readability.

Colour modification is essential for game-ready assets, as it improves user interaction and visual realism.

Screenshot


* MeasuringCup - FreeCAD 1.0.2
File Edit View Tools Macro Drafting Annotation Modification Utilities Windows Help
Draft
Top
2px | 3.50 mm
None
Model Tasks
Model
MeasuringCup
Body
Origin
Sketch
Pad
Pocket
Pad001
Text
ShapeString
Fuse false
Justification Bottom-Left
Justification... Cap Height
Keep Left M... false
Make Face true
Oblique An... 0.00 °
Scale To Size true
Size 10.00 mm
String hayrünisa
Tracking 0.00 mm
View Data
FRONT
MeasuringCup : 1*
Activate Windows
Go to Settings to activate Wind
Preselected: Unnamed1.Body.Pad001.Face1 (-37.53 mm, -13.63 mm, 21.04 mm)
1:1
15
CAD
151.00 mm x 138.05 mm

I added my name - **hayrünisa**



Reflection on Learning

Through this task, I developed practical skills in **CAD modelling**, including sketching, extrusion, hollowing, text customization, and appearance modification. As a beginner in CAD, this exercise significantly improved my understanding of parametric design and how digital objects are created for real-world and virtual applications. The task also strengthened my problem-solving skills and demonstrated how CAD supports modern digital engineering workflows.

References

Autodesk (2023) *Introduction to CAD and 3D modelling concepts*. Available at: https://www.autodesk.com (Accessed: 12 June 2025).

cppreference.com (2024) *C++ reference documentation*. Available at: https://en.cppreference.com (Accessed: 12 June 2025).

FreeCAD (2024) *FreeCAD: Open-source parametric 3D CAD modeller*. Available at: https://www.freecad.org (Accessed: 12 June 2025).

ISO (2020) *ISO/IEC 14882:2020 – Programming Languages — C++*. Geneva: International Organization for Standardization.

Meyers, S. (2014) *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. Sebastopol: O'Reilly Media.

Stroustrup, B. (2018) *Programming: Principles and Practice Using C++*. 2nd edn. Boston: Pearson Education.